25 — SINGLE-LAYER PERCEPTRON FOR AND / OR GATES

Aim

To implement a Single-Layer Perceptron from scratch and use it to learn and classify the **AND and OR logic gates**.

Algorithm + Procedure

1. Define the input patterns for the AND and OR logic gates.
2. Define the corresponding target output values.
3. Initialize the weights and bias to zero.
4. Set a suitable learning rate.
5. For each input pattern, calculate the weighted sum.
6. Apply the threshold activation function to obtain the output.
7. Calculate the error using:

Error = Target − Output

8. Update the weights using the perceptron learning rule:

$$w_i = w_i + \eta \times (\text{Target} - \text{Output}) \times x_i$$

9. Update the bias using:

$$b = b + \eta \times (\text{Target} - \text{Output})$$

10. Repeat the process for a fixed number of epochs.
11. Test the trained perceptron using all input combinations.
12. Display the predicted output for the AND and OR gates.

Code

```
import numpy as np

# Perceptron Training Function

def perceptron_train(X, y, epochs=20, learning_rate=0.1):

    # Initialize weights and bias

    weights = np.zeros(X.shape[1])

    bias = 0

    for epoch in range(epochs):

        for i in range(len(X)):
```

```
            # Calculate weighted sum
            total = np.dot(X[i], weights) + bias
            # Step activation function
            output = 1 if total > 0 else 0
            # Calculate error
            error = y[i] - output
            # Update weights
            weights = weights + learning_rate * error * X[i]
            # Update bias
            bias = bias + learning_rate * error
    return weights, bias
# Prediction Function
def predict(X, weights, bias):
    predictions = []
    for x in X:
        total = np.dot(x, weights) + bias
        output = 1 if total > 0 else 0
        predictions.append(output)
    return predictions
# Input combinations
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
# ---------------- AND GATE ----------------
y_and = np.array([0, 0, 0, 1])
weights_and, bias_and = perceptron_train(
```

```
    X,
    y_and
)
pred_and = predict(
    X,
    weights_and,
    bias_and
)
print("AND Gate")
print("Weights:", weights_and)
print("Bias:", bias_and)
for i in range(len(X)):
    print(X[i], "->", pred_and[i])
# ---------- OR GATE ----------
y_or = np.array([0, 1, 1, 1])
weights_or, bias_or = perceptron_train(
    X,
    y_or
)
pred_or = predict(
    X,
    weights_or,
    bias_or
)
print("\nOR Gate")
print("Weights:", weights_or)
print("Bias:", bias_or)

for i in range(len(X)):
```

```
print(X[i], "->", pred_or[i])
```

Output:

```
AND Gate
Weights: [0.2 0.1]
Bias: -0.2
[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1

OR Gate
Weights: [0.1 0.1]
Bias: 0.0
[0 0] -> 0
[0 1] -> 1
[1 0] -> 1
[1 1] -> 1
```

Result

The Single-Layer Perceptron was successfully implemented from scratch. The perceptron correctly learned and classified the AND and OR logic gates using the perceptron learning rule.